

O'REILLY®

Early
Release

RAW &
UNEDITED



The Agentic Enterprise

A Leader's Guide to Orchestrating,
Governing, and Scaling AI Agent Systems

Babak Hodjat &
Antoine Blondeau

The Agentic Enterprise

A Leader's Guide to Orchestrating, Governing, and
Scaling AI Agent Systems

With Early Release ebooks, you get books in their earliest form—the authors' raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Babak Hodjat and Antoine Blondeau

O'REILLY®

[OceanofPDF.com](https://oceanofpdf.com)

The Agentic Enterprise

by Babak Hodjat and Antoine Blondeau

Copyright © 2026 O'Reilly Media, Inc. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<https://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: David Michelson

Development Editor: Shira Evans

Production Editor: Jonathon Owen

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

August 2026: First Edition

Revision History for the First Edition

- 2026-02-18: First Release

See <https://oreilly.com/catalog/errata.csp?isbn=9798341672512> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *The Agentic Enterprise*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have

used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-67251-2

[LSI]

OceanofPDF.com

Brief Table of Contents (*Not Yet Final*)

Introduction (available)

Part 1: Why the Agentic Enterprise Matters

Chapter 1: From Automation to Agents (available)

Chapter 2: The Business Case for Agentic AI (available)

Part 2: Finding the Right Use Cases

Chapter 3: High-Impact Enterprise Applications (unavailable)

Chapter 4: Evaluating ROI and Risks (unavailable)

Part 3: Architecting the Agentic Enterprise

Chapter 5: Core Capabilities and Technical Foundations (unavailable)

Chapter 6: Designing for Scale and Experience (unavailable)

Part 4: Governance, Trust, and Scale

Chapter 7: Building Trustworthy Systems (unavailable)

Chapter 8: Scaling Without Lock-In (unavailable)

Part 5: Leading Into the Future

Chapter 9: The Road Ahead for Agentic Enterprises (unavailable)

Chapter 10: Checklists, Templates, and Further resources (unavailable)

[OceanofPDF.com](https://oceanofpdf.com)

Introduction

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the introduction of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sevans@oreilly.com.

Humans are Agents. When we say ‘we have agency’ we mean we are able to postulate a goal of our own making, devise an action plan, and execute to deliver on that goal. As agents we have the ability to organize and impact our world, get feedback from our actions, and modify our actions based on that feedback in order to improve.

Machine intelligence has improved such that it is taking a greater share of our decision-making, complementing and augmenting our world as AI agents.

Like AI itself, and most other modern digital technology, we owe the concept of AI Agents to Alan Turing. In his 1950 treatise on **Computing Machinery and Intelligence**, he described machines that perceive input and act toward goals. This definition is silent on the environment within which the agent is operating and perhaps, Turing’s idea was that the agent would be unconstrained to act in the world.

Today, the world in which an AI agent can operate has vastly expanded. Not only do we have a virtual world in the shape of the internet to complement the real world, we can also define any manner of virtual world we can imagine and engineer it into existence through programming.

So which world do AI agents operate in? Is it fair to assume that ultimately an AI agent is to be able to perceive any given environment, and act toward

any given goal? Who defines the goals? Are agents autonomously to decide on which goals to pursue? While this is possible at lower levels, at some point in the hierarchy of agents there needs to be an uber-goal that directs the system and its highest order agents in how they define their goals.

Even if we borrow from nature and define the higher level goal to be survival, we, as humans, are still engineering this into the AI agents.

The same applies to inputs. Surely an AI agent cannot be perceiving the world in its entirety, and so we'd have to engineer sensors for it, designed based on what we believe to be the most important information it would need in its pursuit of its goals.

So AI Agents are an engineering concept.

Through the years, as computing, storage, and AI have progressed, this definition of an AI agent has been adopted, implemented and experimented on in a variety of ways.

In the seventies and eighties, AI scientists started off thinking we could create a single entity that would be artificially intelligent with no need for constraints. This generally capable AI agent would primarily be able to operate in the real world, perceiving any input and acting towards any given goal. This ambitious definition failed, in part due to lack of compute and labeled data resources, and in part because the AI algorithms were simply not up for the job.

In the mid-nineties, the problem of AI was simplified, by simplifying the world within which the intelligence would manifest itself. What if we build an intelligent agent, representing us, not in the complex world we live in, but in the simpler world of the internet, which back then, was mostly text and hyperlinks.

Thus, Agent-based AI took center stage. A popular AI textbook, published in the late nineties, was called "Artificial Intelligence: A modern Approach" and the modern approach was this new constrained agent-based definition of AI.

Before long, people started wondering what would happen if your agent ran into someone else's agent. Should they collaborate or compete? Would there be a common inter-agent language?

Multi-Agent systems became a subfield of AI, with some, including the authors, suggesting a new software paradigm called Agent-oriented Software Engineering (see **Figure I-1**) as a follow on to the object-oriented movement.

non-numerical application of computers, and considered very difficult to solve, have been effectively addressed through the use of LLMs now.

What's more, these AI models can express intent and context in natural language. The very human ability to put concepts together in order to form new concepts, describe new contexts, or express new meaning is now available in machines, which makes them much more robust than any artificial system ever made.

Language was developed by humans as a means for structured, high conceptual bandwidth communications. Without language, human societies would not be functional. Over the years language has become richer as new concepts have emerged. Now that machines 'get' human language, we have a universal protocol for human-to-machine communication -- we can talk to them in our language, and they can respond in kind.

Interestingly, as a consequence of the invention of LLMs, we now also have a much richer and more robust machine-to-machine communication language in the form of our own natural language. Since machines understand English (or German, Chinese or Hindi), we can have them communicate in one or more language, and this has the double benefit of being rich in abstractions and concepts, covering the entirety of humans-developed ontologies, and it has the added benefit that it is understood by us, which is helpful if we want to understand and follow the decision-making process.

So, out of the box, we have a universal protocol for inter-agent communications. Because of the flexibility of natural language to express intent, and the ability of LLM-based agents to map such intent to the manner by which their tools are called, software systems of the future will be much less brittle than the ones we have today: an agent responsible for a functionality takes care of mapping intent to the specific API calls, and so it will be much less of a hassle to change the API, or employ a different software with similar capabilities. This will give organizations more flexibility in upgrades or with third-party services.

If inter-agent communications and coordination are done in a manner that guarantees the encapsulation of responsibilities, then future enterprise applications will be much more robust, with new capabilities and functionality being introduced to a preexisting network of agents with little to no need for reengineering the system that is already there.

Given how a machine learning approach, using deep learning and transformer architectures, and training on language modeling, has resulted in such capable systems as LLMs, it is natural to think that we are back on track to create a single model that is generally intelligent. In other words, some people think that building ever larger LLMs, and training them on more and more data, will result in an LLM that is more intelligent than humans in every aspect of human intelligence. It seems, however, that for very different reasons, we are now back to considering agent-based systems as a next natural step for software (see [Figure I-2](#)).

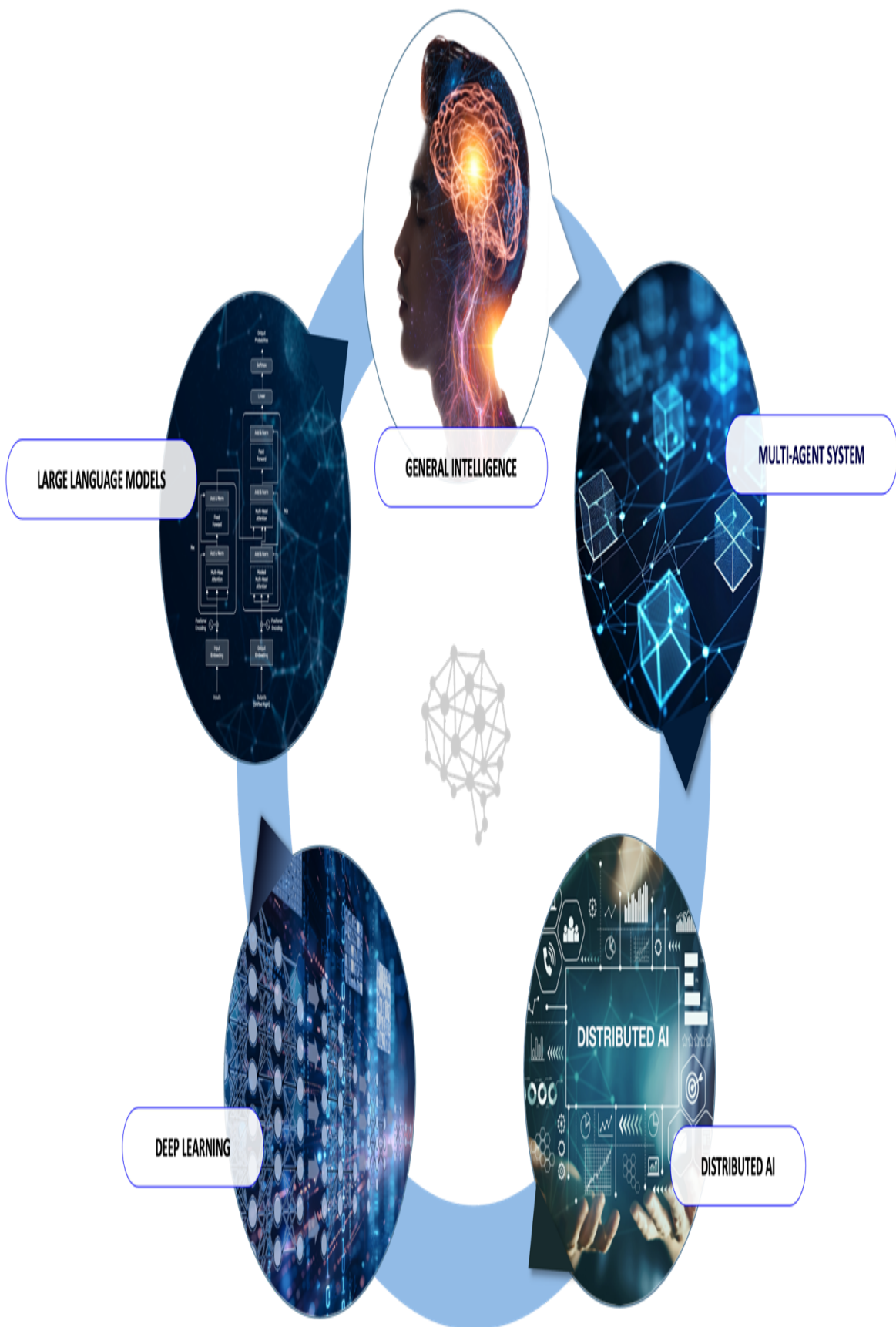


Figure I-2. Agent-based Systems. AI has come full circle: Initial failure in building generally intelligent systems resulted in breaking the problem into smaller sub problems, handled by agents in multi-agent systems, which, in turn, led to distributed AI. Large Neural Networks, or Deep Learning, can be thought of as a Distributed AI system, and LLMs are a special kind of Deep Learning. We make the case that a single very capable LLM cannot be generally intelligent, unless it is used as a multi-agent system.

Generative AI is pre-trained and so it is difficult to make it fundamentally change its world model online. Generative AI is typically trained on publicly available data, so, not having information in its training that is specific to a commercial problem domain, say proprietary data a company may be collecting and using, it is not generally useful out of the box for business applications.

Also, LLMs need to be given a system prompt in order to be useful. A system prompt is part of the LLMs input context that keeps being repeated with every distinct input, so as to remind the LLM who it is and what is expected of it. Through system prompts, LLMs can take different personas and operate differently. This means that, to play a specific role within a knowledge workflow, we need to give an LLM specific instructions, declare it to have specific expertise, and give it a specific job description and set of responsibilities.

The ability of machines to both understand highly complex human-designed concepts and execute specific instructions has led to resurgence in interest in agent-based AI, or agentic AI. Can we get machines to chart their own path towards executing on complex goals, and engineer the set of instructions to make that happen, and do so autonomously?

The challenge is altogether very different to that which the early AI pioneers faced in the nineties: back then, we were forced to simplify the problem domain because of lack of compute resources and model capabilities. Now, we still simplify the problem domain but that is because Generative AI models are too powerful and flexible, and we need to contain them and taskmaster them to specific roles within our organizational use-cases. Thus the advent of modern LLM-based Agents.

Multi-Agent Systems

Many uses of Generative AI call for confining an LLM to a specific role. In figure 3, for example, we could choose to have the LLM respond to the query “What is the average weight of penguins on Biscoe?”. It may already have learned the answer from its general knowledge, but if it has not, it will try to guess, and the answer might not be reliable. But there is a more reliable alternative: if we have access to a database that we know to include reliable data, say on penguins, it is preferred that we tell the LLM about the schema of the database and the type of SQL used to query it, and have it translate our natural language inquiry into a call into the database.

LLMs are great at writing code, but they don’t have the facility to make the actual call and retrieve the data, so we need some helper code for that. Our code would pattern match this intent of the LLM and make the call to the database, retrieve the answer, and pass it on to the LLM to formulate it into a natural language response (see Figure I-3).

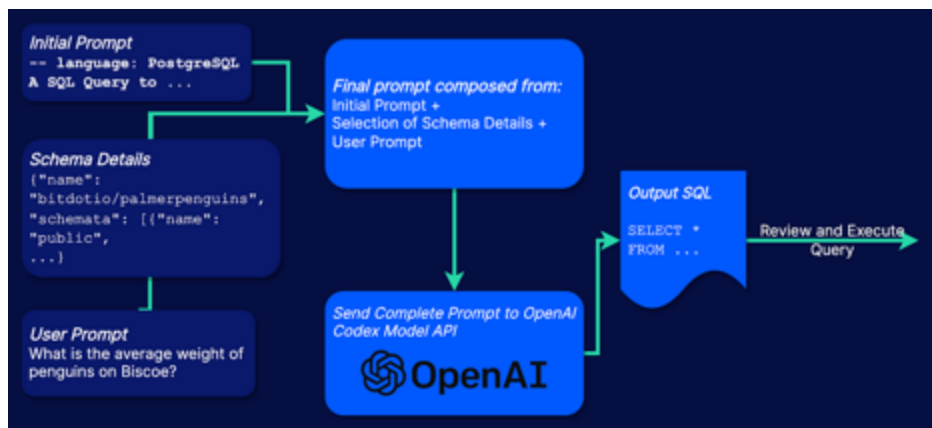


Figure I-3. Using an LLM as an agent to make call to a database.

In fact, a popular use-case of Generative AI these days is Retrieval Augmented Generation (RAG). Many companies are seeking a ChatGPT-like interface to their proprietary document repositories, and this is implemented by summarizing the documents offline and storing them, along with the LLM’s representation of the summaries (known as embeddings), in a vector database. The embeddings are vectors capturing the meaning, or semantics, of the summaries. By storing them in a vector

database, we can calculate numerical distances between the summaries, making it easier to search by meaning, rather than strict and rigid pattern matching of the words and sentences. This system can then respond to future natural language questions regarding the proprietary documents by mapping the requests to queries to the vector database, finding summaries that are closest in their meaning, to the query at hand.

Hence, you can see here too that RAG can be thought of as a multi-agent system, with an agent for offline summarization, and one tasked with responding to the user's natural language queries (see **Figure I-4**).

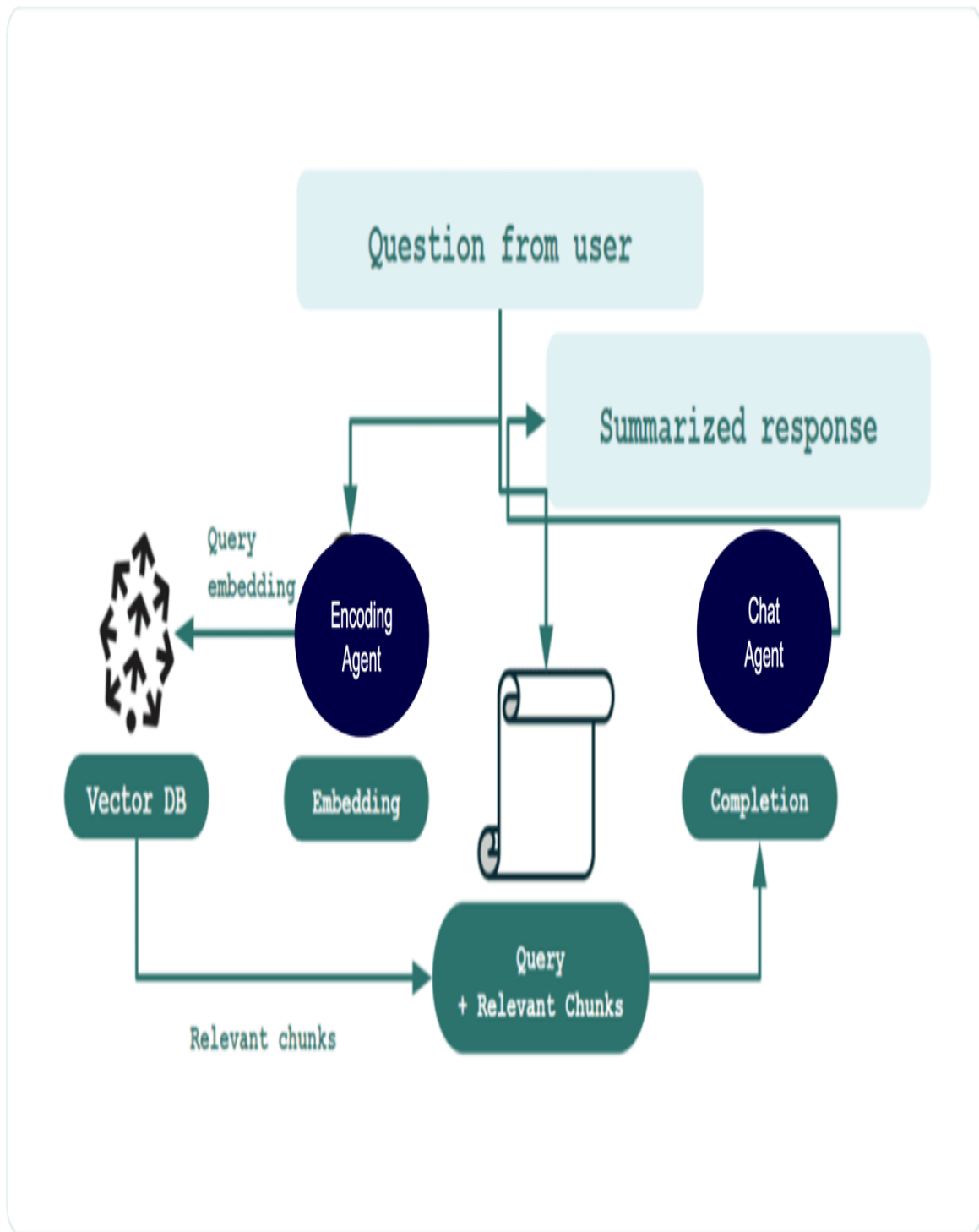


Figure I-4. RAG can be thought of as a multi-agent system.

But much more can be done. Any workflow requiring knowledge workers can be thought of as a multi-agent system. In **Figure I-5**, for instance, we can have a planning agent come up with various plans, say for an upcoming

vacation, an actuation agent with knowledge of APIs for internet services such as Expedia or Hotels.com, can then call the API and get actual numbers, say for airline price or hotel dates, into the plan. And a critic agent can check the results and decide whether they are plausible (e.g., is the connection time for a flight reasonable?) and send them back for refinement if they are not.

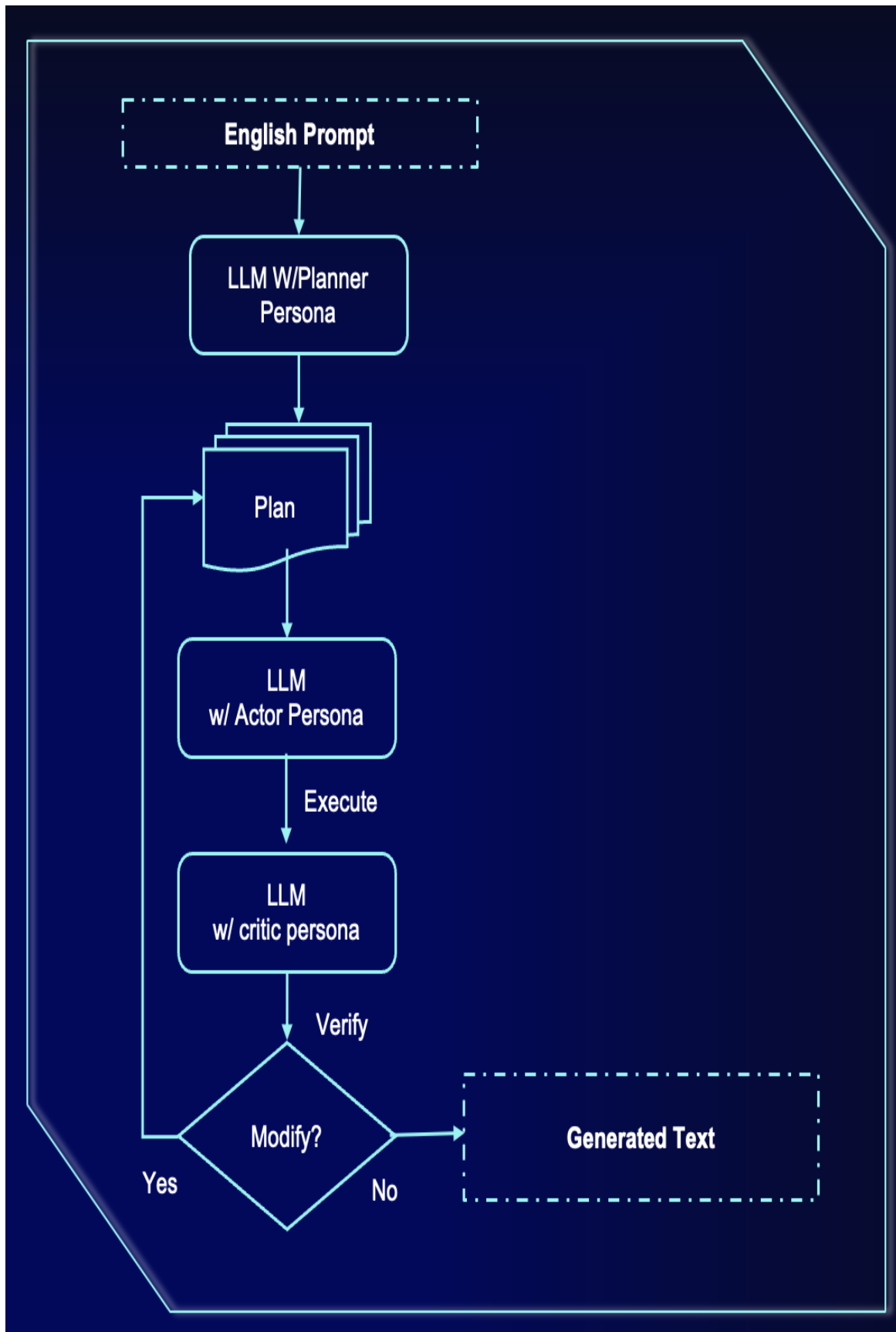


Figure I-5. A Multi-Agent Planning System.

Let's formalize our view for an agent-based system a bit. What we mean by an agent is a Generative AI LLM wrapped around a module, function, service, or database. This allows an end user to interact with that functionality in natural language (see [Figure I-6](#)). It also allows us to log the intent of the agent upon servicing every input from the user. In other words, we can log the why and what of the agent's behavior, because the agent understands and can produce natural language describing every step it is taking. This also allows us to introduce safeguards on the agent's behavior, by having it observed by another agent, let's call it the Safeguard Agent. This agent can be tasked with our ethics, compliance, or regulation guidelines, and intervene if what is asked of the system is not compliant.

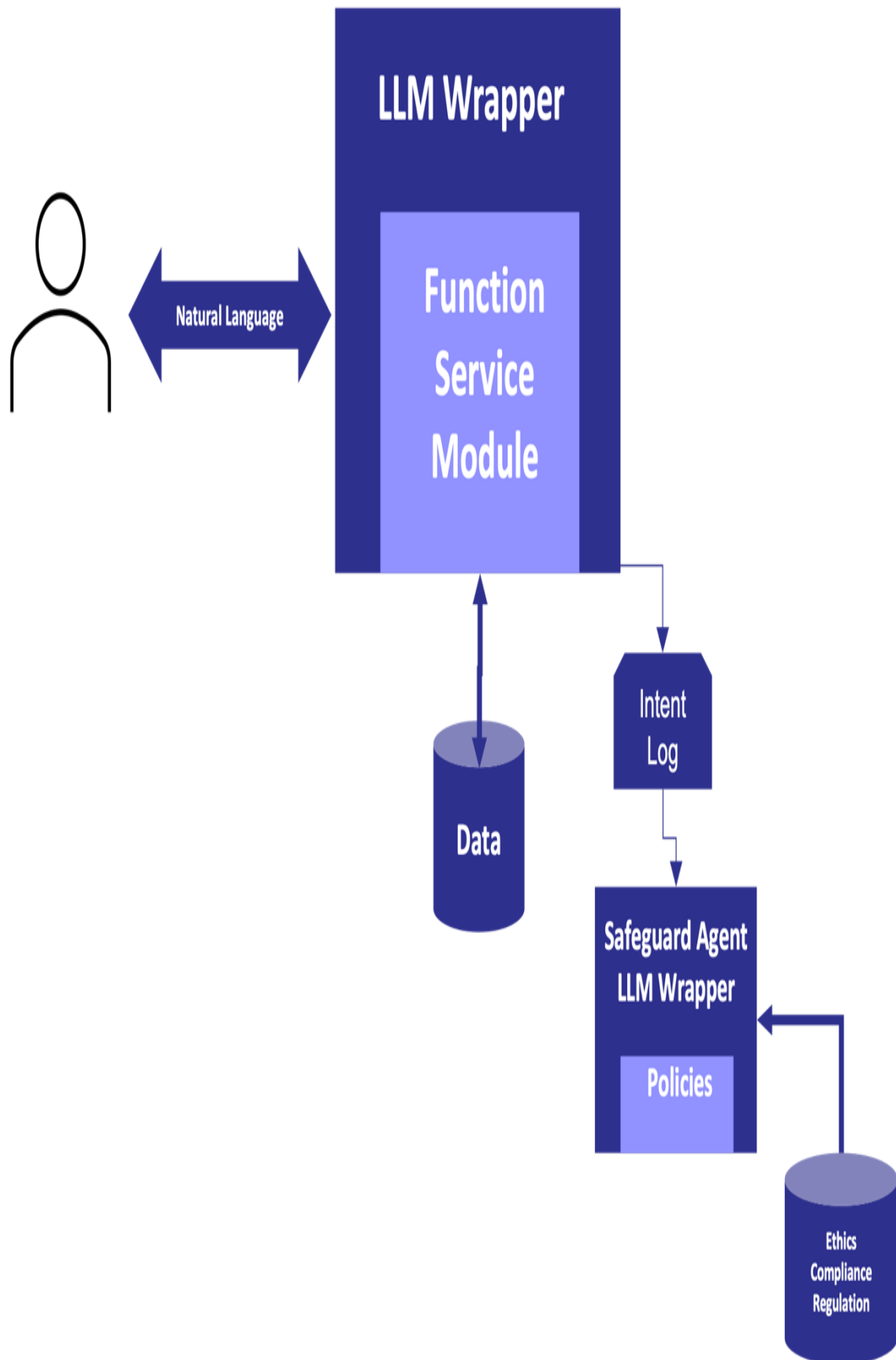


Figure I-6. An LLM agent is an LLM wrapped around some functionality, for instance a database.

By the same token, let's now assume that, in addition to the database in figure 6, our application also includes a microservice (e.g., an API call to a CRM back-end), and some of the user's queries or commands are better handled through that. We have a choice of having the user to separately use an agent for that service, or to have the agent collaborate with another agent that is in direct interaction with the user, making it seamless and more usable.

To make this happen, we can have the original agent servicing the user aware of the agent wrapping the microservice and have them communicate to fulfill user queries. This communication can be in natural language, which means it can capture the intent and be robust to changes in the format and syntax of the specific calls to the underlying services.

Before you know it, an enterprise's entire software and applications can be agentified in this way (see **Figure I-7**).

Figure I-8 will help us clarify the distinction between a model and an agent. A model is a transformation function. It takes data as input, and produces data as its output. A modern-day AI agent consists of, at least, an LLM that has been enabled to call some tools.

A Gen-AI model (or LLM) takes some input and, through the process of predicting what is expected, produces some output. For example, we give it a coding task, and it produces some code, and depending on the complexity of the task, the code may be usable as is, but to run it, you will have to copy-paste it into your programming environment and manually hit ‘run’.

By contrast, an agent, given the right set of tools for coding, would start by having its LLM generate the code, then run it in a container, observe the results, modify the code, and therefore have a better chance of producing useful code. In fact, taking an LLM and running it on a coding benchmark, results in lower accuracy rates, than when the same LLM is powering a coding agent and is given the same coding benchmark.

Models – *generate predictions* based on inputs

Agents – *do things*

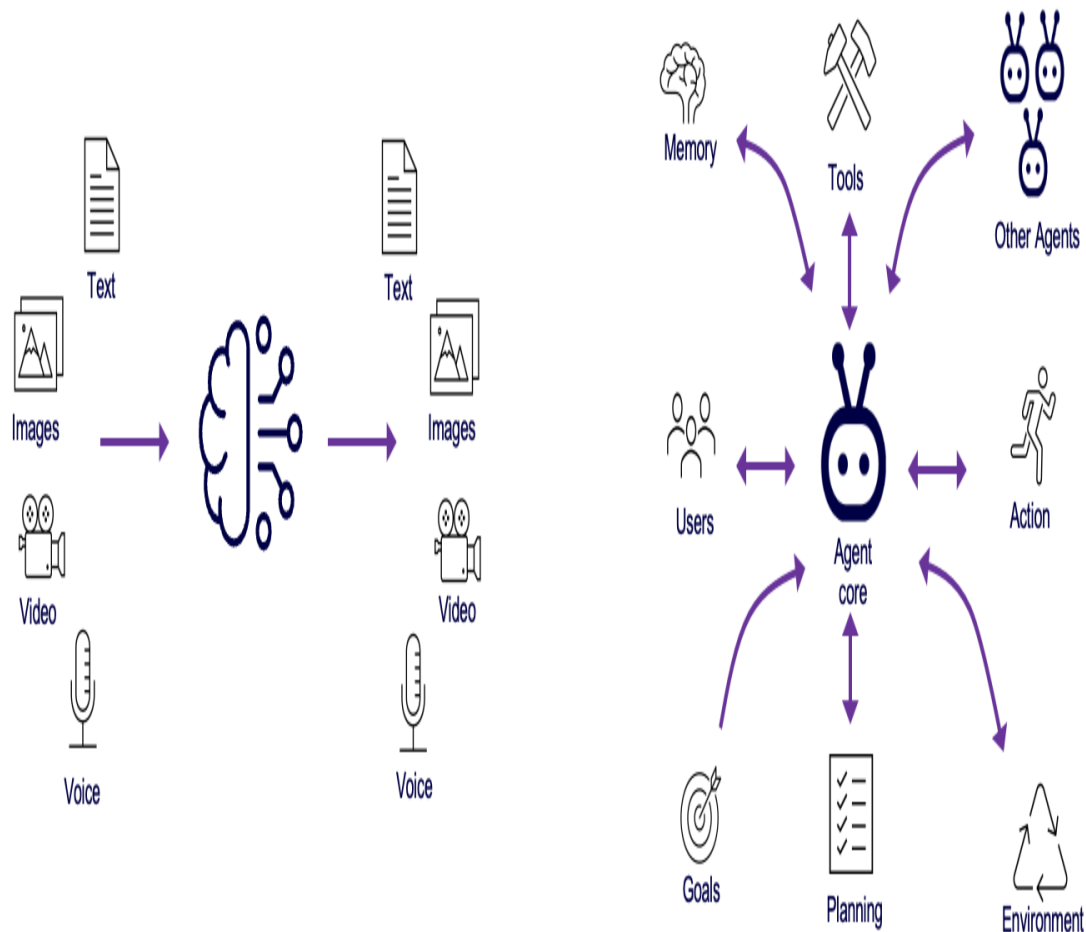


Figure I-8. Model vs. Agents

The field of multi-agent AI calls for agents that can communicate and coordinate. This can be in the context of a team of agents, which implies collaboration, or it may be across teams of agents with varying levels of trust and even adversity.

Agentified Applications and Software

To be effective, agents should be allowed to talk to each other. For example, imagine your company intranet with its useful search box, directing you to the apps and resources you need. If you are a large enough company, these apps, owned by different departments, each have their own search boxes. It makes a lot of sense to create agents, maybe by using techniques like RAG, to augment the search boxes. What does not make sense is to force the user to repeat their query into an app's search box once the intranet search box has identified it as useful given your initial query. Rather, we would prefer the top agent to coordinate with agents representing the various apps and present a consolidated and unified chat interface to you, the user (see [Figure I-9](#)).

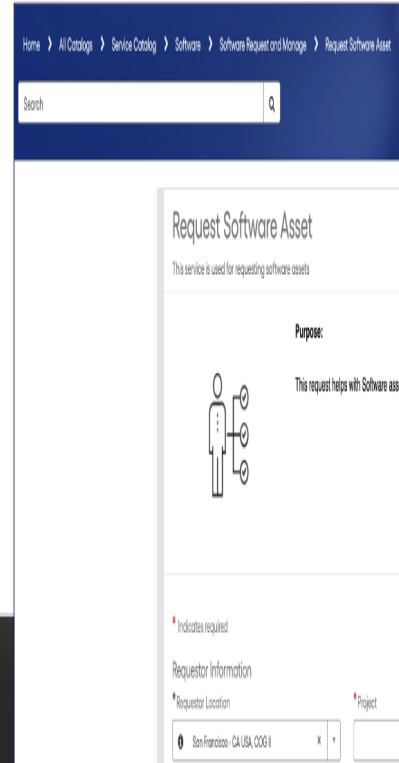
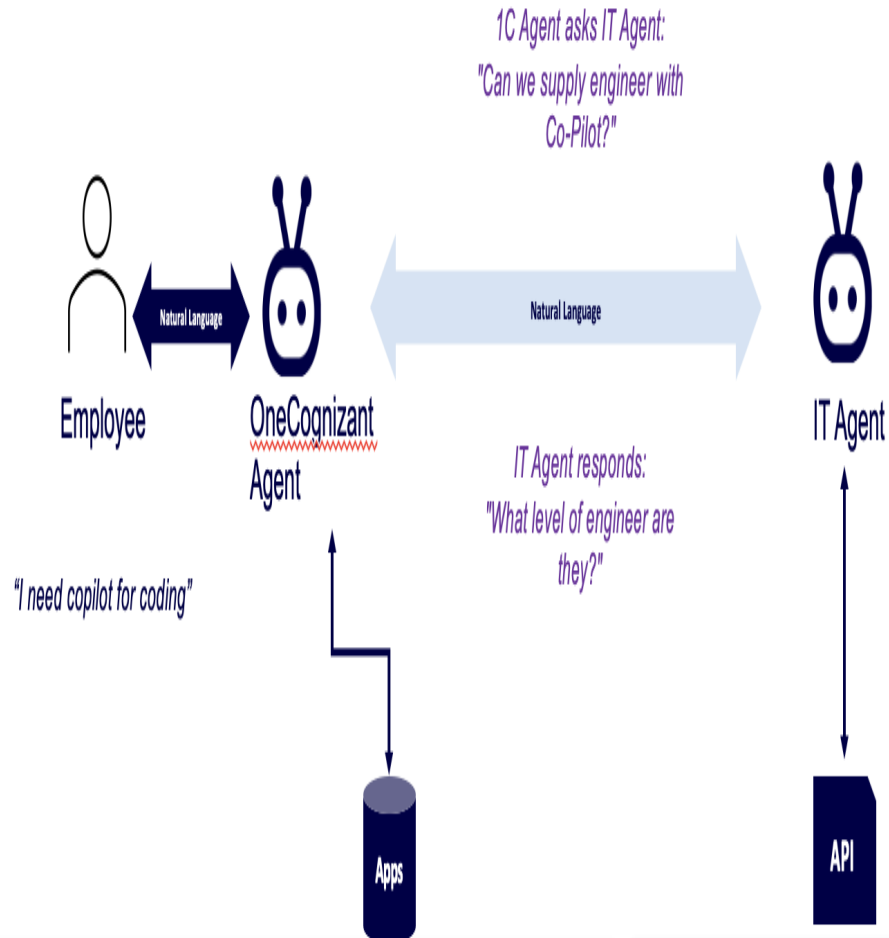


Figure I-9. Agents should talk to one another. Here, we see the agent augmenting the intranet search box talking to the agent augmenting the IT app's search box.

A multi-agent system representing an organization's software or various workflows can have several interesting advantages, including improved productivity, operational resilience, improved robustness, and the ability for faster upgrades of different modules. This book will help you see how this is achieved and provide you with examples illustrating these advantages.

Migrating a software/application stack to an agent-based architecture offers several distinct benefits:

Intent Logging

Each agent can log its intent on every transaction or call it makes to various services, enhancing transparency and traceability. Something that is harder to achieve with software logs, which tend to be terse while voluminous, and lack explanation of the various context and rationale behind the software's behavior.

Robust Inter-Agent Communication

Communications between software and apps represented by agents are intent-based and often in natural language, making the interactions more understandable and reducing the complexity of integration.

Safeguard Agents for Responsible AI

The ability to add safeguard agents to monitor and check the behavior of each agent ensures compliance with responsible AI practices and ethical standards.

Modularity and Flexibility

Agent-based systems are highly modular, allowing for easier updates and maintenance. Each agent can be independently developed, deployed, and scaled without affecting the entire system.

Upgradeability

Software and application modules being represented by agents provide a more seamless and less disruptive way to upgrade the underlying software, as the complexity of mapping intent to the actual back-end calls is handled by the LLMs, which simplifies and even does away with much of the manual rewiring often involved in API and software upgrades.

Interoperability

Agents can interact with diverse systems and services, providing a unified interface and facilitating integration across different platforms and technologies.

Autonomy and Specialization

Each agent can be specialized for a particular task, leading to more efficient and optimized execution of specific functions within the application stack.

Enhanced Fault Tolerance

The failure of one agent or software back-end does not necessarily disrupt the entire system. Agents can be designed to handle failures gracefully, improving overall system reliability.

Improved Data Handling

Agents can handle data locally and only communicate relevant information, reducing data transfer costs and improving performance. Disparate data sources can be represented by different agents, helping mitigate issues with data consolidation or being behind on data modernization.

Better Resource Management

Agents can optimize resource usage by adjusting their operations based on current system states and workloads.

Customizability and Extensibility

New agents can be added or existing ones modified with minimal disruption, allowing for continuous improvement and adaptation to new requirements.

Breaking Software Silos

Redundancies and overlaps in various enterprise software are a constant source of frustration for users. Agents can help reduce or eliminate such silos by operating the software modules on behalf of users, and consolidating and simplifying responses across multiple software modules as they serve a user's intent.

These advantages can result in major operational efficiencies, IT and infrastructure savings, and measurable enhancements to the quality of business decision making.

However, the benefits from multi-agents systems also come with the need for careful and safe design considerations.

Given their role, a level of autonomy is expected of an agent as it interacts with the world using its tools. In some cases, like running code in a container, the consequences can be controlled and contained. In others, scrutiny and care needs to be applied to avoid undesirable consequences of an agent's autonomous behavior.

The fact that the 'brain' of an agent is an inherently opaque neural network (i.e., an LLM) makes this more complex. LLM-based agents are also prone to hallucinations or misunderstanding of inherently ambiguous natural language communications, so what we gain in robustness we can lose in consistency.

Several mitigation strategies are available to reduce these risks, including implementing rules for overriding or seeking human approval for certain agent decisions, assessing uncertainty of agent behavior, and pairing agents

with safeguard agents that monitor and prevent potential harm. Let's take them in turn.

Rules for agent autonomy and control

An agent can be wrapped into deterministic programs that allow us to impose rules on conditions within which the agent is allowed to make use of its LLM's non-deterministic reasoning. The inverse is also possible, where LLM reasoning supersedes rule-based behavior, and we defer to the LLM on when to switch to human defined rule-based behavior.

Using LLM uncertainty

There is a third option to determine when to defer to the LLM's reasoning versus rule-based behavior, or even to call a human to make the decision on what to do. This requires an estimation of LLM uncertainty. While LLMs are opaque or black-box models, which means it is very difficult to determine why they behave the way they do, there are techniques to estimate how confident they are in their own output. For example, it is possible to get a sense of the range of different outputs an LLM is likely to produce for the same input. This range can help us calculate an absolute value for the LLM's confidence for the various outputs it can produce for the same input. If we then set a threshold on this confidence value, we will have a future-proof mechanism to switch between deferring to the LLM, when its confidence is higher than the threshold, and switching to predetermined rule-based behavior when the LLM's confidence does not meet the threshold.

Safeguard Agents

One or more agents can be defined to be responsible to check the proposed behavior of another agent and determine whether the behavior is acceptable or not.

Given the required oversight, we need a different testing regime for agent-based systems that differs from what we are used to in traditional software. The good news, however, is that we are used to testing such systems as we have been operating human-driven organizations and workflows since the dawn of industrialization.

The state of the art in Generative AI models prevents single agents from effectively taking a large body of complex instructions and carrying out too many varied and complex tasks. Given the limit on the size of an LLM's input (i.e., context window restrictions) as well as limitations in the power of reasoning of LLMs, it is often more effective to break responsibilities down into multiple AI agents, connected and coordinating to get work done. Just as we do not assign a single software engineer the task of writing a fully-fledged CRM system, asking an AI Agent to write code for a full CRM system may be a tall order, but having a team of agents representing various responsibilities such as project management, front-end and back-end engineering, and QA, working together is more likely to get the job done successfully.

A modern AI Agent consists of an LLM, and some code. The code allows the LLM to actuate its intentions using the tools we assign to it. Most systems facilitate the calls directly, with no intermediate calls. For example, the Model Context Protocol (MCP), which has become a de facto standard in defining and exposing API for agentic use, allows agents to discover and call API without requiring manual integration. Some agentic systems do allow developers the flexibility to extend the code part of an agent by writing code that would run prior to calling tools (see [Figure I-10](#)).

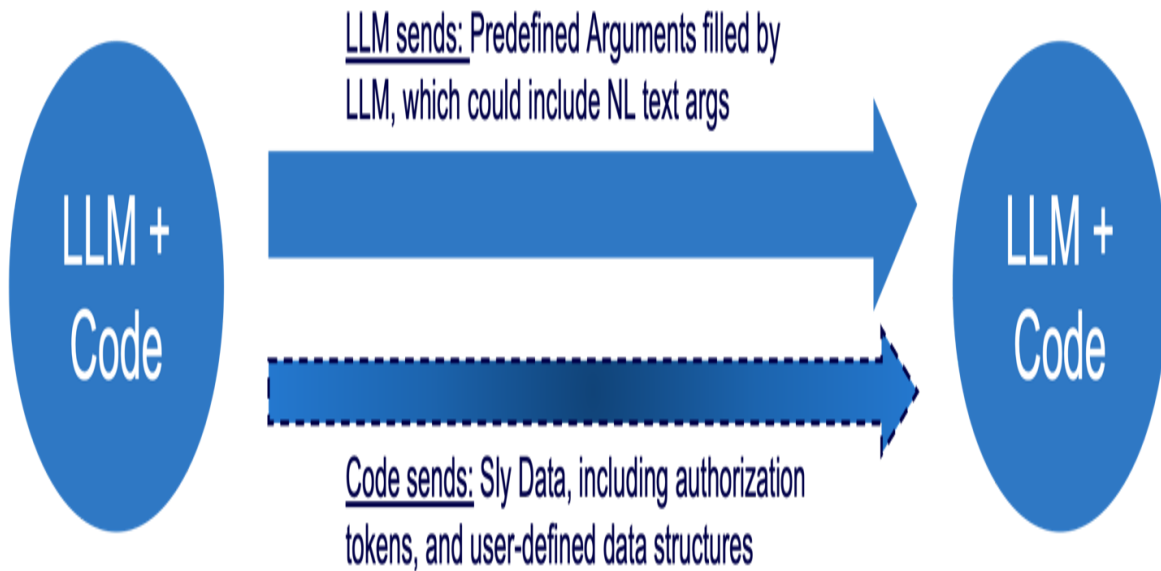


Figure I-10. Inter-agent communications split into code and LLM: the called Agent returns both the agent's LLM generated results, as well as the data structures, which may have been processed by the called agent's tools.

This duality of LLM and code is useful and, as an important step in our design of a multi-agentic system, this allows us to think about what part of an agent's behavior needs to be handled by the LLM, and which part needs to be coded.

There is a tendency, sometimes, to think of AI agents as we did of software modules and try to control them and make them consistent. This is understandable, but defeats the whole purpose of agentification, which requires ceding some control to agents, and accepting some level of inconsistency, in exchange for robustness, efficiency and quality.

When dealing with an agent, we should always assume some autonomy of behavior. An agent should be allowed, part of the time, to decide on its own which of its tools to use, in what order, and how. If you determine that your agent is not allowed any autonomy, then you should strongly consider replacing it with a software module.

Tasks that require reasoning, robust understanding of text or other unstructured input, or the generation of such unstructured data as output should be delegated to the LLM. More rigid rules-based behavior that

requires consistency will have to be implemented in the code part of the agent.

In other words, we should divide an agent's responsibilities between those needing an intelligent knowledge-worker-in-a-box and those requiring predefined rules applied consistently. The former can be delegated to the agent's LLM, and the latter should be programmed into the coded part of the agent.

An essential tool to help apply this division of labor to a multi-agent system is `sly_data`: a data structure that can be operated upon by the code part of an agent, and that is passed around between agents through code, and so is not necessarily subject to LLM processing.

What also appears clearly is that, beyond the technical means (like `sly_data`) to orchestrate multi-agents in an enterprise, a need is emerging to essentially define some sort of Governance of the agents among themselves and of their interactions (or not) with humans. To some extent, the orchestration of multi-agents mimics human enterprise organization rules and how different specialized teams and units interact with each other.

The Agentic User Experience

As Agentic AI straddles the concepts of autonomy in automation and “human in the loop” control, we see three important considerations in realizing the transformative potential of multi-agent systems when it comes to user experience: first, perception of delay or latency in processing, second, the multiplicity of interaction channels and choices for interactions, and third, the need for allowing human intervention when required.

Delay/Latency perception

Despite the objective performance gains of multi-agent systems, their intrinsic processing can create a perception of delay compared to the near-instantaneous feedback of traditional software. Strategically reducing this perceived latency is essential for frictionless user

integration and the full realization of multi-agent systems' innovative potential.

Multiplicity of interaction channels and choices

Multi-agent systems offer new and expanded interactions choices: with the advent of multi-modal LLMs as the brains behind the agents, multiple interaction modalities are made possible. With multiple agents responsible for various domains and functions within an enterprise, users can be offered a variety of choices for interacting with different agents. These requests could be for asking for updates and information, or in the form of commands to affect change.

Human-in-the loop interventions

Responsible design of multi-agent systems also calls for some critical implementations to be human-in-the-loop, or, for some systems that can afford to run with more autonomy, to operate in a human-on-the-loop mode, allowing humans oversight and the ability to intervene, as well as tasking with the agents to pull humans in when and wherever needed.

Each of these aspects call for special UX considerations and best-practices. Ultimately, superior user experience in a multi-agentic enterprise should allow for easy and natural entry points and interaction modalities.

Multiple entry points

Regardless of whether the user persona is a customer, or an employee, a modern multi-agentic enterprise should allow access to agents from a variety of entry points and respect a variety of modalities. Users should be able to switch entry-points and mix modalities, and the system should be able to recognize the user intent and context through a combination of implied and expressed cues.

Channel agnostic interactions

Just as we are able to use various means to interact with our colleagues, interaction with the multi-agentic enterprise should also be channel

agnostic, and users should be able to use a chatbot or search box on the application's web page, a mobile app, or corporate chat apps like Slack and Teams. Often, the interactions are not chat-based at all. The context of applications, data inflows, and browser-based interactions could all be inputs to an agentic system, triggering various behavior and multi-agentic processes.

Direct interaction with chosen agent

Where possible, users should be allowed to initiate their interactions with the agent they think would be responsible for handling their request. For example, a corporate intranet consisting of agents for functions like HR, Finance, Legal, etc., should not only be able to handle queries and commands directed at the top agent, but it should also allow the user to go directly to the agent the user feels is the most relevant to the topic. If the user knows, for example, that their request is related to HR, they should be allowed to start their conversation with the HR agent.

Multi-Agent Organizations

A multi-agent system representing an organization's various workflows can have several significant advantages, including improved productivity, operational resilience, and improved robustness. For businesses that are rapidly moving to adopt single agent solutions multi-agent systems can appear to be an inevitable and quite disruptive future state.

Why use multiple specialized agents communicating with one another, versus using a single all-powerful, know-it-all agent? Essentially, using multi-agents offers a number of benefits over using single agents: flexibility in incremental engineering of complex systems, ability to safeguard an agent's behavior, improved agent's reliability, and broader choice of LLMs to use:

Flexibility in engineering

From an engineering perspective encapsulating responsibilities and implementing a distributed inter-agent coordination mechanism (e.g., AAOSA) means that we can build these agent networks incrementally, plugging in new agent sub-networks without having to re-engineer existing agents.

Safeguarding agent's behavior

Safeguarding an agent's behavior is more effective when we pair the agent with a Safeguard agent, versus asking the agent to do something, but to be careful at the same time.

Better agent reliability

From a consistency and reliability perspective the approach assumes much more specialization for each agent, making agents much more reliable in their behavior and less prone to issues related to LLM confabulation and hallucination.

Broader choice of LLMs to use

This also means that the bar is a bit lower on what LLM to use depending on the agent. Larger more capable agents are expensive, slow, and mostly hosted, and being able to use smaller more specialized LLMs means you are paying less, getting quicker responses, and having the option to host agents that handle more sensitive data, and to have an easier time fine-tuning the LLMs, if needed. Moreover, and as LLM context is limited, even for more powerful models boasting millions of tokens as their input context, a single agent for all uses can hit limitations. On the other hand, if context is used wisely, each agent in a multi-agent setting remains much more focused and less reliant on recalling and respecting what it is asked.

The same goes for output. If you ask an LLM to produce a complex output, it is likely to fail at producing everything you requested, whereas for an

agent in a multi-agent setting, restricting responsibilities typically results in less output complexity requirements.

However, it is important to note that the larger the number of agents, the more costly and slow the system, so there is a balance to be struck between modularity, robustness, and consistency on the one side, and cost and response time on the other. Care needs to be taken as to the granularity of the agentification, as well as the types of LLMs being used and when and how to fine-tune them to make them effective. The latter is an option, once an initial multi-agent system is in production or running in a sandbox, where usage data can be collected, labelled by human auditors as to acceptable or not, and this labelled data then used to fine-tune the LLMs behind various agents so as to make their behavior more acceptable.

Given the power of multi-agent systems, one can expect that the fabric of future organizations will consist of interconnected agents; however, for the foreseeable future, and unless a seismic shift occurs in culture and society and agents are conferred some level of personhood, humans will remain the sole bearers of responsibility. This means that multi-agency should be controlled and designed with humans as the ultimate arbiter and approver of higher risk behavior.

Chapter 1. From Automation to Agents: The Evolution of Enterprise AI

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sevans@oreilly.com.

Most early enterprise applications of AI were rule-based expert systems, with human designed taxonomies helping with business decisions, and in the case of RPA (Robotic Process Automation), automating some steps in a rigid process.

With the advent of Deep Learning, large neural network models trained on data to help with classification and prediction problems, some experimentation and niche use-cases started, with the hopes that such models would uncover patterns in the data that had remained hidden to statistical analysis.

It must be mentioned here that, prior to the introduction of LLMs, the field of applied AI had a much more diverse set of tools in deployment. These included symbolic regression, evolutionary optimization, predicate logic systems, random forest models, to name a few. Today, some of these approaches are not even recognized as AI, and applied AI is mostly LLM-

based. Approaches such as Deep Learning are commonly referred to as ‘traditional’ ML models now and LLMs have overshadowed the variety of AI tools and approaches in the field.

While we acknowledge the fact that the field of AI is much richer and wider than LLMs, in this book, we will define AI Agents using their more recent configuration, which always includes an LLM, wrapped in a program that actuates the LLM’s intent.

November 2022 was a pivotal moment for AI. ChatGPT 3 was released and suddenly, a much wider audience got an idea for how profoundly powerful and generally applicable AI systems can be. With all its flaws, ChatGPT 3 was able to ‘understand’ any command or query, attempt to respond to it, and explain itself in clear natural language.

The form by which this powerful AI was revealed has had a lasting effect on how it has been deployed: a chatbot interface for retrieving data and information, best suited for support calls, FAQs, knowledge repositories, and general help.

OpenAI, and others, were also quick to point out an interesting behavior of LLMs: confabulation. These models seemed to make things up, what is commonly known as ‘hallucination’. This, paired with the fact the LLM models pull from a probability distribution and are therefore inconsistent in their behavior, raised many flags with businesses looking to deploy AI. There was also a data security consideration due to the fact that the most powerful LLMs were hosted by commercial entities like OpenAI, Google, and Anthropic. It didn’t help that OpenAI announced it would store user queries and use them to improve its AI (they later retracted this).

Businesses therefore initially mostly focused on low risk, high return use-cases, and ended up scratching the surface of applicability. The two main application areas outside of personal employee productivity, were RAG-based information retrieval, and coding companions (i.e., copilots).

High expectations on AI, yet limited use-cases, meant that the initial expected ROI on LLM-based AI deployments went unmet.

Another factor beyond the limited application areas was the expectation that AI systems, being generally intelligent, would be deployable out of the box, with very little if any customization. This is exacerbated by the fact that most AI labs were started and got their funding to produce AGI (Artificial General Intelligence), which implies a monolithic, one-size-fits-all AI model. It took the world a year or two to realize that applied AI should be engineered and customized to the use-case.

Today, there is still a case to be made as to why we should engineer multi-agent systems versus relying on a single LLM-based agent to do everything.

To start with, the manner by which single-agent systems are being fine-tuned on Chain of Thought reasoning (CoT) is useful for certain agentic tasks requiring reasoning or coding, but it does seem to bind the agent to a certain average reasoning behavior that reduces its general applicability. In using reasoning models for coding, these systems often get stuck into a certain manner of “thinking” and start repeating themselves.

The moment we move from using models (e.g., LLMs) to using LLM-based Agents, we are entering engineering territory. The manner by which an agent’s responsibilities and tools are defined and customized for a use-case requires elaborate design and verification. This is all the more important when we are designing multi-agent systems as the interconnection and roles of the various agents needs to be engineered. For the same use-case, the design of an agentic system can make the difference between effective and safe versus ineffective and risky. The job description of the agent, and the selection and implementation of its tools depend very much on the use-case. Modularization, with its benefits (reuse, specialization, and divide and concur), is well suited for engineering work.

There is no one-size-fits-all LLM for all agents. This becomes even clearer considering the advantages of using smaller, more specialized LLMs for more specialized agents. This gives us the option to run some agents on-prem, for example in settings where there are sensitive data considerations.

Given our experience with LLMs as chatbots, it is often thought that agents are transactional systems that do not do anything until invoked by a text

command, and go back to sleep after they have responded. Agents, however, can be set up to wake up on other non-user events. Some agents are run constantly, checking their environments and context, proactively communicating with users or other agents, or being on a long term mission of discovery, maintenance, or improvements.

For example, coding agents can open pull requests and react to events like reviewer comments, or code merges. In a telco setting, some agents might be constantly running on network nodes on load balancing and incident recovery missions.

Another practical consideration that makes us lean towards multi-agent systems is interoperability. Many businesses are promoting their own agents and enterprises will want to connect these agents to each other, or to internally connect proprietary ingrown agents with one another.

You can also fine-tune LLMs depending on their job description within the larger multi-agent system. Simply collect and label data specific to an agent's behavior and fine-tune for each agent separately. How would you do that with a single agent that is meant to do everything?

Context size is still a limiting factor for LLMs. Even with LLMs that can take a relatively large number of tokens as input, staying faithful to everything that is stated in the context is a challenge. In agentic systems, the system prompt, dialog history, memory, and tool descriptions all take valuable context tokens. All these elements are reduced as we limit and focus the responsibility of individual agents within a multi-agent system. If we are smart about memory and dialog history, we can afford to be much more selective and specific about how we use an agent's context, making agents more reliable and predictable in their behavior.

Then there's the question of performance. Is a single LLM-Agent more efficient than the equivalent multi-agent system? A recent position paper argued for how more refined decomposition into agents is critical for (at least) efficiency at scale [<http://arxiv.org/abs/2502.04358>]. There's even some evidence that scaling laws analogous to those in neural networks, which resulted in the amazing emergent behavior in LLMs, are also

exhibited in multi-agent systems, and we are talking cooperation among more than 1000 agents here [<https://arxiv.org/abs/2406.07155>].

Finally, there is a more fundamental question behind this choice between human engineered systems of AI Agents, versus autonomously AI-Engineered systems: Where do we, as humans, exert our control and design? Or are we just simply expressing intent and leaving the rest to the LLM?

The answer is not as obvious as our initial intuition. We are building increasingly intelligent systems that, in many ways, are already smarter than us. Would we rather have a less intelligent system (us humans) design and set up more intelligent systems (AI Agents)? We are already seeing spectacular performance by large reasoning models on benchmarks many of us would have a very hard time acing. Isn't it time for us to defer to these models as the artificial brains for handling all aspects of a system's operations? Let's just give all the APIs for a system to a single very powerful LLM and let it decide how and when to use them. Afterall, we are still in charge because we are expressing our intent as input to the agent.

Not so fast. There are fundamental limitations to state of the art AI, which should give us pause here. Our LLMs have a hard time knowing what they don't know (and for that matter, they also have a hard time knowing what they do know). In other words, they are lacking meta-cognition, something that seems to come quite naturally to us, perhaps because our brains are wired differently than a feed-forward, train-once, run many times, LLM.

Also, while seeing AI generated artwork, prose, or even poetry does demonstrate incredible creativity by LLMs, another paper we recently coauthored shows how current ML and reinforcement learning methods have a hard time dealing with the unexpected effects of creative actions, or what is commonly referred to as 'unknown unknowns' [<https://arxiv.org/abs/2501.13075>].

These limitations are not easy to overcome, unless we rethink our approach to AI or we have some major breakthroughs.

Multi-agent systems allow us to engineer safeguards for such limitations. Rather than potential magnification of risky behavior and inconsistencies due to relying solely on a single monolithic LLM-Agent, we can build resilience in distributed, modularized, redundant, multi-agent systems. These systems can have built-in agentic checks and balances as well as safeguards, including rule-based and human fallbacks. Additionally, the multi-agent approach allows us to build and extend systems incrementally, testing and fine-tuning various agents in isolation and within a sandboxed larger system, before plugging them into the live agent network. It also provides room for regulators of responsible AI to mandate the use of third-party agents for more critical and risky operations.

[OceanofPDF.com](https://oceanofpdf.com)

Chapter 2. The Business Case for Agentic AI

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sevans@oreilly.com.

Undeniably, ChatGPT and Large Language Models (LLMs) have raised awareness of AI in an unprecedented way, thanks to the ease of natural language interaction and the impressive (if sometimes imperfect) outputs they can generate at the touch of a button.

However, this recognition has quickly been followed by three broad sets of concerns. First, bias and hallucinations have undermined the perceived quality of outputs that initially looked striking. Second, the compute power required to run these models has raised questions about their environmental impact. Third, financial analysts have highlighted uncertainties around the returns on the massive investments made in the LLM ecosystem. In other words, and perhaps unfairly, the initial excitement and curiosity around LLMs have given way to skepticism about the potential for large-scale, economically rational corporate adoption. While Agentic AI relies on LLMs for, for example, document processing, data retrieval, workflow instructions, API management, and compliance tracking – and because it requires not just efficiency but also accuracy and compliance – Agentic AI

has had to mitigate LLMs' flaws through guardrails, governance, and system design to achieve superior, goal-oriented outcomes.

As a result, Agentic AI has moved quickly beyond the “interesting” stage to become a strategically urgent and important technology, driven by both market maturity and competitive pressure. It is transforming AI from tactical automation into a more strategic, value-creating capability. This, in turn, requires corporations to rapidly assess their organizational and technical readiness for Agentic AI if they want to capture its benefits.

Market maturity and competitive drivers

Three converging forces make this the right strategic moment for Agentic AI and turn what was a bleeding-edge experiment into a mainstream enabler: advances in technology, economic pressure, and organizational maturity.

Firstly, the underlying technologies for Agentic AI have already passed a clear inflection point. Foundation models and orchestration layers have demonstrated that agents can operate autonomously across a range of domains, such as customer service, supply chain and logistics, and compliance. This level of maturity is underscored by the fact that agents are already running key processes in highly regulated industries like insurance (claims, underwriting) and banking, where regulators are stringent about protecting customer interests and avoiding systemic risk. In parallel, the broader Agentic AI infrastructure has matured at record speed – through cloud-native APIs, secure model hosting, and related capabilities – giving sophisticated organizations confidence that adoption will meet their professional and regulatory standards. Two real-life examples come to mind that showcase the transformative power of Agentic AI implemented at scale for mission-critical processes. Take Allianz Partners, reporting that they have scaled autonomous AI with Otera's agentic workflows – moving beyond PoCs to a true operational backbone with up to 90% automation of eligible claims, settlement times cut from around 30 days to 3–4 days, and a path from €300M to €1B in profitability by 2030, according to their Chief

Data Officer. Another financial-services leader, Franklin Templeton, reports a similar magnitude of impact on its core business in collaboration with Wand AI. Citing the press release on this collaboration: “Over the past year , Franklin Templeton has deepened its collaboration with Wand AI, evolving from pilot programs to full-scale production deployments of intelligent agents across multiple departments. The initial implementation focused on priority use cases within the firm’s investment teams. By 2026, Wand-powered agents are expected to be used in key enterprise workflows – supporting investment research, operational efficiency, and digital transformation – all underpinned by rigorous governance and control frameworks.”

Secondly, from an economic standpoint, the timing for Agentic AI is also right. Over the past decades, most corporations and public-sector organizations have undergone multiple waves of automation via various IT systems and solutions. The most recent waves before Agentic AI focused on specialized Robotic Process Automation (RPA), which excelled at automating scripted, rule-based, repetitive tasks but did so in a rigid manner. Some RPA leaders even branded certain solutions as “agents” working alongside human agents on specific tasks. Yet these early RPA solutions had clear limitations that Agentic AI is now addressing: being truly goal-oriented (deciding *how* to achieve a goal), demonstrating the level of autonomy and adaptability needed to do so, triaging complexity (and escalating to humans when appropriate), and orchestrating interactions with other agents and systems to reach an outcome. For example, Allianz Partners USA had used RPA for more than a decade to process its claims more swiftly, only to replace – in 2024 – its RPA with AI-powered technology capable of auto-paying suitable claims, while Allianz Group is deploying multi-agent systems (for example, under Project Nemo) that orchestrate coverage checks, weather data, fraud analysis, and payouts end-to-end. RPA automated steps, and Agentic AI now automates the outcome.

While focused RPA implementations eventually exhausted their opportunities for marginal efficiency gains, Agentic AI injects new momentum into problem-solving by introducing decision velocity –

coordinating multiple levers in a largely autonomous way. This new momentum has already translated into impressive double-digit cost savings for large global organizations, as shown above, across their most important middle- and back-office processes.

However, front-office tasks, such as lead generation, are also being reshaped by Agentic AI solutions that drive significant revenue improvements, which bodes well for stronger and more sustainable ROIs. Take Artisan.AI, a San Francisco-based start-up that has developed an agentic Sales Development Representative (SDR) that automates outbound prospecting by finding, researching, and prioritizing B2B leads at scale. It continuously scans context signals about a company – such as its industry, the software it uses, recent hiring patterns, or new funding rounds – to identify accounts that are more likely to be ready to buy, before any human salesperson gets involved, then runs personalized, multi-step email and social outreach, so sales teams get a steady flow of warm, context-rich leads instead of cold lists.

Thirdly, organizational awareness and adoption readiness have increased markedly. After the emergence of LLMs and the initial “wow effect” experienced by employees and managers across industries, the question of “what should we actually do with this?” quickly rose on senior leaders’ agendas. In many organizations, LLM adoption has been cautious – starting with “wait and see” positions until implications were better understood, followed by a flurry of proofs of concept that often did not make it into production. Yet this period has not been wasted: the idea that AI is inevitable has now crystallized in leadership circles, and many organizations have used this exploratory phase to build foundational capabilities in data and governance (MLOps, data governance) that create fertile ground for Agentic AI to take root.

Ultimately, there is a growing recognition among corporate leaders that Agentic AI is more than a marginal tool. Citing Allianz Partners’ Chief Data Officer on their deployment of Agentic AI: “We didn’t have time to do proof of concepts, play around with generative AI, but rather think of this as part of our backbone in our organization to perform tasks, to make

decisions [...] This was the very first time you could really hand over decisions. Once we launched the Otera agents... this was really a moment where I think the whole company realized the autonomous AI era is here.”

Strategic value vs. tactical automation

Given that automation technologies have been used in business for years, it is legitimate to ask where Agentic AI truly adds value. A useful way to answer this is to contrast two automation archetypes: tactical automation and strategic (agentic) automation.

Tactical automation assumes linear processes and rule-based decisions. It has been used primarily to reduce costs by replacing repetitive human effort in middle- and back-office processes. Once these processes have been fully automated, the scope for additional gains tends to be limited and exhibits diminishing returns.

Strategic (agentic) automation, by contrast, is non-linear and adaptive. Here, value comes from decision leverage – the fact that autonomous decision-making offloads a significant amount of work from humans, who are brought into the loop only by exception. In this model, ROI is driven by the agent’s ability to discover the operating context, decide the “next best action,” and build an emergent form of cognition that enables adaptive personalization and continuous optimization.

For example, consider an AI agent handling a flight-booking request. If the airline website returns an error message, a traditional system would often stall, but an agent can decide what to do next: try another path on the same site, switch to a different airline, or alter parameters such as dates or airports – without getting stuck. This illustrates how goal-oriented automation differs from narrow task execution.

This leads to a useful three-layer view of value in automation:

- Value layer 1 – Operational productivity

Value from automating tasks and speeding up execution – productivity gains (tasks are completed faster and at lower unit cost),

- Value layer 2 – Outcome quality and resilience

Additional value from agentic coordination across domains, leveraging autonomous decision-making towards achieving an overall goal – better outcomes (decisions are more consistent, better-informed, and more robust across scenarios),

- Value layer 3 – Adaptive, enterprise-wide systems

Further value from enterprise-wide adaptive systems that learn in context – adaptiveness gains (new business models emerge and tap new value pools).

In layer 3, entirely new models can arise. In consumer retail, for instance, agentic commerce platforms may treat AI shopping agents as the primary “client,” not the human buyer, forcing product providers to rethink how they market, position, and price their offerings to machines as much as to people.

In this context, agentic systems are particularly well-suited and value-adding in certain types of use cases, such as:

- Customer support, where agents proactively resolve issues instead of reactive chatbots simply escalating frustrated users to a human,
- Supply chain optimization, where agents dynamically reroute shipments and adjust inventory levels,
- Compliance, where agents monitor regulatory changes and continuously self-audit the organization against updated rules.

Conversely, agentic systems may create less value – or introduce unacceptable risk – in certain situations, for example:

- Environments where human judgment is highly subjective and individualized (“I like this jacket, but I want the fabric to feel softer”),
- Scenarios where critical information is tacit, fragmented, or impossible to fully encode for an agent, such as complex M&A negotiations, major credit decisions on opaque counterparties, or crisis-management situations where material facts reside in people’s heads or informal networks rather than in systems of record,
- High-stakes contexts where the cost of a mistake outweighs operational benefits (for example, misinterpreting a compliance rule that could trigger a multi-million-dollar fine, or altering settings on a life-support device).

Organizational and technical readiness

Agentic systems show strong prospects for value creation and, at the “value layer 3” level, open the door to significant market and competitive disruption through new business models. Business leaders should view these opportunities – and the corresponding risk of being disrupted – as a wake-up call to accelerate their organizational and technical readiness, which is a key driver of the business case for Agentic AI.

This readiness can be framed across six dimensions:

1. Leadership alignment

- Do we, as a leadership team, understand the core agentic concepts and their strategic implications – both as opportunities and as threats?
- Have we clearly identified the business pain points or opportunities that are worth addressing with Agentic AI?

- For those areas, have we defined clear boundaries between agent autonomy and necessary human intervention?

2. Technical infrastructure

- Do we have the necessary infrastructure – data, MLOps, APIs – to support our Agentic AI agenda?
- To what extent can we still move forward if parts of our infrastructure are not yet ideal, and where are the non-negotiable gaps?

3. Governance and risk

- Are we equipped with documented principles to guide our stance on Agentic AI – for example, how to balance autonomy with augmentation of human agents?
- How will we ensure these principles are implemented, monitored, and enforced?
- What existing or new committees should oversee these questions, and who is accountable when agents act autonomously?

4. Architecture

- How will we design modular systems and their orchestration, while ensuring seamless interaction with legacy systems?
- How will we prioritize which systems to modernize first, and which can wait, given budget and resource constraints?

5. Technical resources availability

- Do we have the right technical skills in-house to design solutions, learn from feedback, and iterate effectively?

- Do we have the skillsets needed to understand the underlying models and deploy agentic scenarios in a friction-free manner?

6. Change management

- Do we have the right talent, cultural agility, and risk appetite to take advantage of Agentic AI?
- Are we prepared to evolve roles, skills, and ways of working as agents become part of core processes?
- What does it mean for our organization to manage both humans and machine-based agents as a unified “workforce”?

For each dimension, leaders can use a simple RAG (Red/Amber/Green – not to be confused with Retrieval-Augmented Generation) assessment – Red (we are behind), Amber (we are progressing but not there yet), and Green (we are on top of this, or leading) – to gauge current maturity. This helps prioritize efforts and sequence investments in a pragmatic, business-driven manner.

Agentic Systems – What is the Payoff?

Ultimately, the business case for Agentic AI can be summarized in the following key points.

Maturity

Agentic AI technologies have reached a level of maturity where they can serve as mainstream business enablers rather than experimental add-ons.

Opportunity

Agentic systems create opportunities for strategic – not merely tactical – impact, shifting the focus from incremental efficiency to

transformational effectiveness and adaptiveness.

As a result, business leaders must quickly assess how opportunities (or negative developments) could arise from adopting – or choosing not to adopt – such technologies.

Assessing organizational and technical readiness should be approached with a practical mindset: pacing the effort, being explicit about where and how Agentic AI can benefit the enterprise and determining appropriate guardrails. As these technologies evolve rapidly, cultivating enterprise-wide awareness and agility enables a rational approach to value creation while preserving strategic options for the future.

[OceanofPDF.com](https://oceanofpdf.com)